

AI Model Registry Project Report

Student Name: Adarsh Ramakrishna

Student ID: A00336108

Date: 26 November 2025

1. Introduction

Managing AI software at scale is a real and growing challenge. Companies today maintain dozens sometimes hundreds of distinct AI models, each created by a different team, serving a different purpose, and built with a different architecture. Without a centralised system to keep track of them, things get messy fast: teams duplicate effort, metadata gets lost, and governance becomes nearly impossible.

The AI Model Registry is a solution to a big problem. It's a special tool, built using Java, that helps keep track of AI models in a organized way. Think of it like a digital notebook where users can add new models, include important details like who made it and when, and group them by type - like text, images, or audio. This registry is better than just keeping a list in a file or trying to remember everything manually. It has a special interface that makes sure all the information is consistent and accurate, so bad data can't get in and cause problems. This way, users can rely on the registry to have the right information, every time.

The main goal of this project was not just to create something that functions, but to do it in a way that's thoughtful and intentional. When making design decisions, we wanted to use the latest features of the Java language, follow good Object-Oriented Programming rules, and write code that's easy for someone else to understand and work with, even if they're seeing it for the first time. We chose to use Java 21 and Java 22 because they offer better, safer, and more straightforward ways to solve problems that are tricky to handle with older Java syntax. This approach makes the code cleaner and more efficient.

2. User Stories

The application was shaped by a set of practical user requirements, each representing a real need that the system should fulfil:

- **US-1:** As a user, I want to register a new AI model so that it is saved in the system.
- **US-2:** As a system administrator, I want to prevent models with invalid data such as negative parameter counts from being entered into the registry.
- **US-3:** As a manager, I want to view a complete list of all models currently held in the registry.
- **US-4:** As a developer, I want to filter the model list to locate specific entries for example, models whose names begin with a particular letter.
- **US-5:** As a user, I want the training duration for each model to be calculated automatically based on the model type, so I do not have to input it manually.
- **US-6:** As a user, I want creator name and creation date metadata attached to every model record, ensuring traceability.

3. Architecture

The class structure is organized in a clear hierarchy, with the `AIModel` sealed class acting as the foundation. This root class defines a shared contract that applies to all models, which includes having a name, a creation date, a creator, and a type. Building on this foundation, the `LargeLanguageModel` class extends the root class and adds its own set of domain-specific properties, such as `parameterCount`, to provide more detailed information. This hierarchical structure allows for a logical and systematic organization of the classes, making it easier to understand and work with the different models. The `AIModel` class serves as a base that provides a common set of characteristics, while the `LargeLanguageModel` class adds its own unique features to provide a more specialized set of capabilities.

The Computable interface introduces a second axis of design, separate from inheritance. It defines a `runInference()` method that any implementing class must provide.

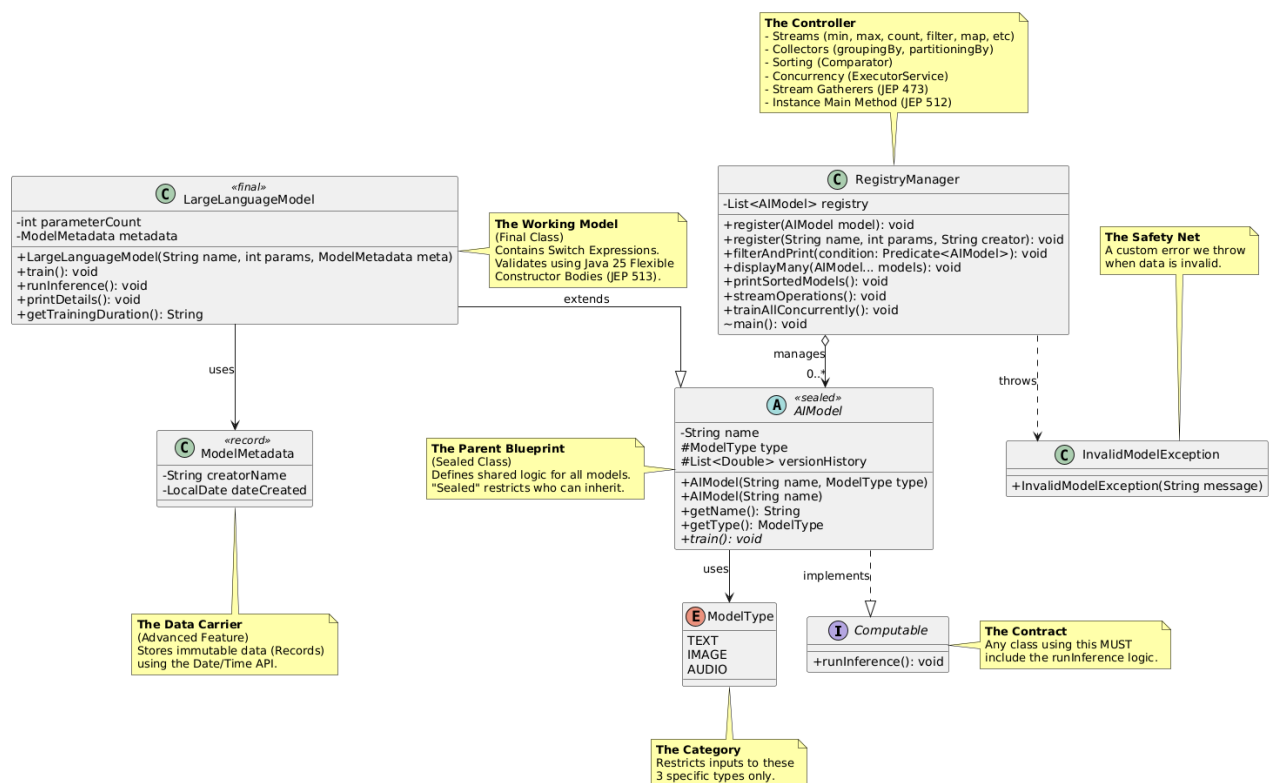
Supporting classes include:

ModelMetadata : a Java Record holding the creator name and creation date

ModelType : an Enum defining three valid categories: TEXT, IMAGE, and AUDIO

RegistryManager : the central service class for storing, validating, and querying models

InvalidModelException : a custom checked exception thrown on invalid registration



4. Fundamentals

4.1 Sorting — `Comparator.comparing()`

Sorting can be done with a simple statement that uses a method reference to extract the key. This means you don't have to manually implement a `Comparator`. If you need to sort by multiple keys, you can chain them together using `.thenComparing()` without having to rewrite any of the logic. It's a pretty straightforward way to get things in order.

java

`registry.stream()`

`.sorted(Comparator.comparing(AIModel::getName))`

`.forEach(System.out::println);`

4.2 Lambdas — Core Functional Interfaces

Four interfaces from `java.util.function` are used throughout:

- `Predicate<AIModel>` — filters models by any condition (name prefix, parameter count, type) without changing `RegistryManager`
- `Consumer<AIModel>` — performs side effects like printing; powers `forEach()`
- `Supplier<AIModel>` — lazily creates a default model when the registry is empty
- `Function<AIModel, String>` — transforms models to strings; drives `.map()` in streams

4.3 Streams

Streams replace imperative loops with readable, single-purpose pipelines.

Terminal operations: `count()`, `min()/max()`, `findFirst()/findAny()`,
`allMatch()/anyMatch()/noneMatch()`, `forEach()`

Collectors: `toMap()` for fast lookup; `groupingBy()` for type-based grouping;
`partitioningBy()` to split into two groups by predicate

Intermediate operations: `filter()`, `map()`, `distinct()`, `limit()`, `sorted()`

java

```
Map<ModelType, List<AIModel>> byType =  
    registry.stream().collect(Collectors.groupingBy(AIModel::getModelType));
```

4.4 Switch Expressions & Pattern Matching

Switch expressions yield a value directly and enforce exhaustiveness, removing the `break/return` ceremony of traditional switches. Pattern matching combines `instanceof` checks and casting into a single inline declaration.

java

```
return switch (this.modelType) {  
    case TEXT -> "2 weeks";  
    case IMAGE -> "4 weeks";  
    case AUDIO -> "3 weeks";  
};
```

```
if (model instanceof LargeLanguageModel llm) {  
    System.out.println("Parameters: " + llm.getParameterCount());  
}
```

4.5 Sealed Classes

`AIModel` is declared `sealed`, restricting subclassing to only the types listed in `permits`. This prevents unauthorised extensions at compile time, protecting assumptions made elsewhere in the codebase.

java

```
public sealed class AIModel permits LargeLanguageModel { }
```

4.6 Date/Time API

Using `LocalDate` is a better choice than the old `java.util.Date` because it can't be changed and it's easier to format with `DateTimeFormatter`.

java

```
String formatted = LocalDate.now().format(DateTimeFormatter.ofPattern("dd MMM  
yyyy"));
```

4.7 Records

`ModelMetadata` is a Java Record — one declaration line replaces a full class with fields, constructor, getters, `equals()`, `hashCode()`, and `toString()`. Immutability is enforced by the type.

java

```
public record ModelMetadata(String creatorName, LocalDate createdOn) {}
```

5. Advanced (Assignment 2)

5.1 Concurrency — **ExecutorService** & **Callable**

Running training models one after another is a waste of time because the computer's brain is just sitting there doing nothing. But if you use a team of three workers to do all the tasks at the same time, it can save a lot of time. Instead of taking as long as all the tasks added together, it only takes as long as the longest task. This is done using a special tool called **invokeAll()**.

java

```
ExecutorService executor = Executors.newFixedThreadPool(3);
```

```
List<Future<String>> results = executor.invokeAll(tasks);
```

```
executor.shutdown();
```

Callable is preferred over **Runnable** because it can return a result and throw checked exceptions.

5.2 Instance Main Methods — JEP 512 (Java 25)

The main method in Java is not needed anymore. Instead, you can use a simple instance method and the launcher will find and call it for you.


```
java
```

```
void main() {  
  
    // rest of my code  
  
}
```

5.3 Flexible Constructor Bodies — JEP 513 (Java 25)

You don't have to call `super()` as the first line in a constructor anymore. Now, validation happens before the parent class is initialized, which means you can stop things from going wrong right away. This is known as fail-fast design and it's a good thing.

```
java
```

```
LargeLanguageModel(String name, long parameterCount) {  
  
    if (parameterCount < 0) throw new IllegalArgumentException("...");  
  
    super(name);  
  
    this.parameterCount = parameterCount;  
  
}
```

6. Extra Marks — Stream Gatherers (JEP 485)

Java 25 has a new feature called `Gatherers.windowFixed(n)` that helps with fixed-size windowing in the middle of a pipeline. This means that models can be grouped into sets of three, which is useful for things like limiting how many requests an API gets or handling output that's split into pages.

```
java
```

```
registry.stream()
```

```
.gather(Gatherers.windowFixed(3))
```

```
.forEach(batch -> System.out.println("Batch: " + batch));
```

5. Source Code Repository

The complete source code, commit history, and version control records for this project are publicly accessible on GitHub.

Repository: <https://github.com/adarsh6980/AI-Model-Registry-Using-JAVA/tree/main>

6. Problems Encountered and Solutions

Understanding Sealed Classes

Early in development, an attempt was made to create a second subclass of `AIModel` without adding it to the permits list. The compiler rejected this immediately. Tracing through the error message made the design intent of Sealed Classes click: they are not just a restriction but an explicit declaration of trust. The permits list is the developer saying "I know exactly what extends this class, and nothing else should." Once that mental model was in place, the feature became a useful tool rather than an obstacle.

Exception Handling and Code Execution Order

A subtler issue emerged during testing. The filtering logic was initially written after the line that intentionally triggered `InvalidModelException`. Because a thrown exception immediately exits the try block, none of the filter code ever ran. The fix was straightforward once the cause was identified: reorder the code so that all successful operations including filtering execute before the deliberately invalid registration is attempted. This reinforced an important principle: the sequence of operations in a try block matters, and anything that must run should not be placed downstream of a line that is expected to throw.

7. Conclusion

The AI Model Registry has come a long way since its inception, undergoing significant transformations across two major assignments. Initially, a robust foundation was laid using object-oriented programming principles, including the use of sealed classes to exert architectural control, records for handling immutable data, predicates for flexible filtering, and the Date/Time API for accurate temporal tracking. Building on this foundation, the second assignment introduced a range of tools that enabled the application to scale more effectively. This included the use of streams for expressive data queries, parallel training with `ExecutorService`, and the integration of three Java 25 features designed to reduce boilerplate code and enhance constructor safety. By incorporating these features, the AI Model Registry has become more efficient, scalable, and better equipped to handle complex data queries and processing tasks. The evolution of the registry is a testament to the power of iterative development and the importance of building on a solid foundation to create a robust and scalable application.

Each new feature was added because it was really needed. For example, streams replaced the old way of doing things with loops, which was prone to errors. Concurrency was added to fix a specific performance issue. Flexible constructor bodies filled a gap in Java's validation model that had been there for a long time. Stream Gatherers made it possible to process things in batches, which was something that could only be done with

custom code before. The end result is a registry that makes sense, is easy to add to, and is designed to catch common mistakes early on - either when the code is being compiled or when it's first run. This means that mistakes are found and fixed sooner, rather than later, which makes the whole system more reliable.